UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN

ASIGNATURA: Metodologías de Desarrollo de Software

NRC: 30693

TEMA: Fundamentos de la Ingeniería de Software y Producto Software

AUTORES:

- Ayala Eatan
- Caizaluisa Katerine
- Cuichán Esteban
- Catota Johan

PARCIAL: Primer Parcial

DOCENTE: Ing. Jenny Ruiz

FECHA DE ENTREGA: 13 de abril de 2026

SANGOLQUÍ – ECUADOR

FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento: Ciencias de la Computación
	Carrera: Tecnologías de la Información Asignatura: Metodología de Desarrollo Software NRC: 30693

INDICE

1. ANTECEDENTES.....3

2.OBJETIVO.....3

3. DESARROLLO.....4

4. CONCLUSIONES 22

5. RECOMENDACIONES.....22

6. BIBLIOGRAFÍA 23

7. ANEXOS (opcionales) 23

8.TABLA DE ESTUDIANTES 24

FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento: Ciencias de la Computación
	Carrera: Tecnologías de la Información Asignatura: Metodología de Desarrollo Software NRC: 30693

1. ANTECEDENTES

- Como parte del aprendizaje de la materia se solicitó analizar y responder las preguntas del capítulo 1 del libro "Fundamentos de Ingeniería de Software", permitiendo analizar los conceptos iniciales de la ingeniería de Software y comprender su importancia en el desarrollo de software.

- El docente de la asignatura solicito la realización de la autoevaluación del capítulo 2 del libro "Fundamentos de Ingeniería de Software", teniendo como objetivo reforzar la comprensión de los conceptos presentados en el capítulo.

2. OBJETIVO

2.1 Objetivo General

Analizar y desarrollar las actividades correspondientes al capítulo 1 y 2 del Libro "Fundamentos de Ingeniería de Software un enfoque práctico", con el fin de comprender sus conceptos, principios y aplicación en el desarrollo de sistemas.

2.2 Objetivos Específicos

Identificar los conceptos fundamentales de la ingeniería de software y su importancia en el desarrollo de sistemas informáticos.

Analizar los principales retos, paradigmas y modelos de calidad en la ingeniería de software.

3. DESARROLLO

Actividad del Capítulo 1

1. ¿Qué es ingeniería de software?

La ingeniería de software no se limita únicamente a la escritura de código (programación), sino que integra procesos, métodos y herramientas para el desarrollo de sistemas complejos que sean confiables, eficientes y económicos.

Según Sommerville (2011) , se diferencia de las ciencias de la computación en que estas se enfocan en las teorías y fundamentos, mientras que la ingeniería de software se centra en la practicidad de desarrollar y entregar software útil.

"La aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento de software; es decir, la aplicación de la ingeniería al software" (IEEE, 2017).

2. ¿Cuáles son las actividades fundamentales de la ingeniería de software?

- Desarrollo: Creación y construcción de sistemas de software
- Operación: El funcionamiento del software en máquinas reales
- Mantenimiento: Acciones para conservar el software a lo largo del tiempo
- Diseño: La planificación y estructuración de los programas de computadora.
- Documentación: La creación de la información asociada requerida para el desarrollo y operación.

3. ¿Cuál es la diferencia entre ingeniería de software y tecnologías de la Información?

La Ingeniería de Software esta principalmente enfocada en el diseño, desarrollo, prueba y mantenimiento de programas y sistemas informáticos usando métodos y procesos estructurados. Su propósito principal es crear software de calidad que resuelva problemas específicos de los usuarios o de las organizaciones.

Esta disciplina aplica principios de ingeniería para planificar proyectos de software, analizar requerimientos, diseñar arquitecturas de sistemas, programar y realizar pruebas para asegurar que el software funcione correctamente.

La Ingeniería en Tecnologías de la Información (ITIN) se centran en el uso, administración y mantenimiento de los sistemas tecnológicos que permiten almacenar, procesar y transmitir información dentro de una organización.

Las personas que trabajan en TI suelen encargarse de administrar redes, servidores, bases de datos, seguridad informática y soporte técnico para garantizar que la infraestructura tecnológica funcione correctamente.

4. ¿Qué es software?

Se define el software como conjunto de instrucciones que permite ejecutar tareas específicas en una computadora. Está compuesto por una estructura que facilita la manipulación de la información y por una documentación que describe el proceso y el uso del sistema.

Según Roger Pressman (2010), el software es una configuración de elementos que debe ser tratada bajo un enfoque de ingeniería sistemático, diferenciándose de un simple proceso

artesanal al buscar atributos de calidad como la mantenibilidad, confiabilidad y eficiencia para resolver problemas del mundo real.

5. ¿Cuáles son los atributos del buen software?

- Alta calidad: Debe ser construido como un producto que cumpla con estándares superiores.
- Confiabilidad (Fiabilidad): El software debe ser confiable y seguro en su funcionamiento.
- Eficiencia: Debe trabajar de manera eficaz en máquinas reales.
- Rentabilidad: Su desarrollo y obtención deben realizarse de forma económica y rentable.
- Robustez: Debe ser capaz de satisfacer las necesidades complejas del mundo real.
- Seguridad: Es un factor crítico, especialmente destacado en las tendencias de la última década.
- Mantenibilidad: Debe permitir su operación y mantenimiento sistemático a lo largo del tiempo.

6. ¿Cuáles son los principales retos que enfrenta la ingeniería de software?

La ingeniería de Software enfrenta diversos retos en los últimos años, los más destacable son:

Gestión de la complejidad e invisibilidad: Se refiere a la necesidad de aplicar un enfoque sistemático para evitar proyectos fallidos debido a la naturaleza abstracta y compleja del software.

Heterogeneidad de sistemas: La necesidad de desarrollar software que se adapte a distintos dispositivos, sistemas operativos y todo tipo de entorno tecnológico e informático.

Seguridad y Confianza: Enfatiza a la necesidad de la protección de datos y la robustez frente a ciberamenazas, constituye a una prioridad crítica desde hace más de una década.

Entrega rápida y competitividad: La alta demanda del mercado presenta un reto de adoptar metodologías agiles y practicas DevOps, sin comprometer la calidad y la robustez del sistema.

7. ¿Cuáles son los costos de la ingeniería de software?

Los costos en la Ingeniería de Software corresponden a todos los recursos económicos necesarios para desarrollar, implementar y mantener un sistema de software a lo largo de su ciclo

de vida. Una gran parte de estos costos está asociada al trabajo del personal especializado, como analistas, desarrolladores, diseñadores y responsables de pruebas, ya que el desarrollo de software depende principalmente del conocimiento y el tiempo de los profesionales que participan en el proyecto.

Además, los costos también incluyen los recursos tecnológicos utilizados durante el desarrollo, como computadoras, servidores, herramientas de programación, licencias de software y plataformas necesarias para crear y ejecutar las aplicaciones. A esto se suman los gastos relacionados con la gestión del proyecto, la planificación, la documentación y la coordinación del equipo de trabajo.

Otro aspecto importante es el mantenimiento del software después de su implementación, que implica corregir errores, realizar actualizaciones y adaptar el sistema a nuevos requerimientos o cambios tecnológicos. Debido a esto, el mantenimiento puede representar una parte significativa del costo total de un sistema de software a lo largo del tiempo.

8. ¿Cuáles son los mejores métodos y técnicas de la ingeniería de software?

- Programación estructurada: Técnica creada entre las décadas de los setenta y ochenta para organizar el código.

- Desarrollo orientado a objetos: Surgió en los setenta y ochenta, extendiéndose en los noventa a las fases de análisis y diseño.
- Métodos Ágiles: Implementados para reducir la complejidad de metodologías previas, siendo Scrum el método que vive un auge actualmente.
- Métodos Híbridos: Buscan el equilibrio combinando la adaptabilidad de los métodos ágiles con la formalidad y documentación de los métodos rigurosos.
- Lenguaje de Modelamiento Unificado (UML): Utilizado para el modelamiento de sistemas.
- Proceso Unificado de Desarrollo (RUP): Primer proceso de desarrollo diseñado específicamente para la orientación a objetos.
- Enfoque sistemático y cuantificable: Aplicación de una disciplina de ingeniería al desarrollo, operación y mantenimiento del software.

9. ¿Qué entiende por paradigmas de la ingeniería de software?

Se entiende como paradigmas de ingeniería de software al conjunto de métodos, herramientas y procedimientos que son adoptados por un equipo de desarrollo para la construcción de software de manera sistemática. No es simplemente técnicas, sino una estructura de pensamiento que dirige cómo se planifica, se diseña y se construye el sistema para resolver

problemas específicos. Estos paradigmas han evolucionado desde enfoques rígidos y secuenciales hacia modelos más flexibles y adaptativos en respuesta a la demanda actual del mercado.

10. ¿Cuál es la definición del modelo incremental versus otros paradigmas?

Características y diferencias con otras ingenierías.

El **modelo incremental**, según Roger Pressman y Bruce Maxim (2020), consiste en desarrollar software en **etapas sucesivas (incrementos)**, donde cada una entrega una parte funcional del sistema. El primer incremento incluye las funciones básicas (producto núcleo), y los siguientes se construyen a partir de la retroalimentación del usuario hasta completar el sistema.

Paradigma	Enfoque Principal	Relación con el Modelo Incremental
Cascada	Lineal y rígido. No se pasa a la siguiente fase hasta terminar la anterior.	El incremental es más flexible, ya que permite ver resultados funcionales antes del final del proyecto.

Prototipado	Se construye un modelo rápido para que el usuario lo vea.	Mientras el prototipo puede ser desechable, el incremento es una parte real y permanente del sistema final.
Espiral	Basado fuertemente en el análisis de riesgos.	El incremental se enfoca en la entrega por partes; la espiral puede incluir incrementos, pero prioriza la gestión de riesgos en cada ciclo.

Tabla 1 Paradigmas Desarrollo Software

Características clave:

- Entrega temprana de software funcional.
- Prioriza los requisitos más importantes.
- Permite incorporar cambios con relativa facilidad.
- Divide el sistema en módulos que se integran progresivamente.

Diferencias con otras ingenierías:

- El software es **inmaterial y flexible**, permitiendo entregar partes funcionales antes de terminar todo.

- Los cambios son **posibles y menos costosos** que en áreas como la ingeniería civil.
- Cada software es **único**, no se produce en serie, por lo que el modelo incremental ayuda a reducir la incertidumbre.

Actividades Capitulo 2 (Autoevaluación)

1. ¿Qué es una metodología de desarrollo de software?

Un conjunto de lenguajes de programación que permiten analizar, diseñar y construir productos software.

Un conjunto de rutinas de programación que permiten desarrollar aplicaciones de forma ágil.

Un conjunto de métodos que cubren todo el ciclo de vida de desarrollo de sistemas y que están unidos por un enfoque general filosófico.

Justificación: Una metodología de desarrollo de software no se limita a utilizar lenguajes de programación o rutinas de código, sino que establece un conjunto organizado de métodos,

procesos y principios que guían todo el ciclo de vida de un sistema. Esto incluye etapas como el análisis de requisitos, el diseño, la implementación, las pruebas y el mantenimiento. Por lo tanto, una metodología define cómo se debe desarrollar el software, cómo se estructura el trabajo del equipo y qué prácticas se deben seguir para asegurar calidad y eficiencia en el proyecto.

2. Las actividades estructurales del proceso de desarrollo de software son:

Despliegue/comunicación/modelado/diseño.

Gestión de la configuración/modelado.

Administración del proyecto, diseño.

Justificación: Las actividades estructurales se justifican porque el producto software es el resultado de un ciclo de vida con actividades debidamente estandarizadas, secuenciadas y estructuradas para cumplir con los requisitos del cliente. Este proceso abarca desde el levantamiento de requisitos (comunicación) y la planificación sistemática (modelado/diseño) hasta la entrega del producto funcional (despliegue), garantizando así un enfoque disciplinado en la creación, operación y mantenimiento del sistema.

3. Una de las razones por la cual se incrementan los costos al agregar funcionalidad

después que un sistema está en operación es:

Lenguaje de programación seleccionado.

Edad de los programadores.

Práctica de desarrollo deficiente.

Justificación: Cuando el software se desarrolla sin buenas prácticas de Ingeniería de Software, el sistema suele quedar mal estructurado, con poca documentación, código difícil de mantener o sin una arquitectura clara. Esto provoca que, cuando el sistema ya está en funcionamiento y se desea agregar nueva funcionalidad, los desarrolladores deban modificar muchas partes del sistema, corregir errores o reestructurar el código, lo que incrementa significativamente el tiempo y el costo del trabajo.

4. Ingeniería de software asistida por computador significa:

Conjunto de herramientas de software integradas para ser utilizadas por otras.

Conjunto de herramientas de hardware integradas para ser utilizadas por otras.

Elementos físicos del computador integrados para ser utilizados por otras.

Justificación

La Ingeniería de Software Asistida por Computador (CASE) se refiere a un ecosistema de aplicaciones diseñadas para aumentar la productividad y la calidad en el desarrollo. Estas herramientas automatizan actividades del ciclo de vida.

De acuerdo con Sommerville (2011), las herramientas CASE "proporcionan apoyo automatizado a las actividades del proceso de software", lo que permite a los ingenieros gestionar de manera más eficiente la complejidad inherente a los sistemas informáticos.

5. La metodología con sus tipos correspondientes de clasificación es:

Metodología Scrum (estructurada).

Metodología XP (orientada a objetos-ágil).

Proceso unificado de desarrollo (estructurada).

Justificación: La metodología XP (Extreme Programming) es un enfoque ágil centrado en la adaptabilidad y la entrega continua de software funcional. Está orientada a objetos porque aprovecha principios de la POO para facilitar cambios rápidos, refactorización y modularidad.

Agilidad: entrega temprana de software y adaptación constante a cambios en los requisitos.

Orientación a objetos: uso de encapsulación y polimorfismo para crear componentes reutilizables, mantenibles y fáciles de probar.

Manifiesto Ágil: https://agilemanifesto.org/iso/es/manifesto.html

6. En gráfico propuesto, la secuencia ordenada de las figuras es:

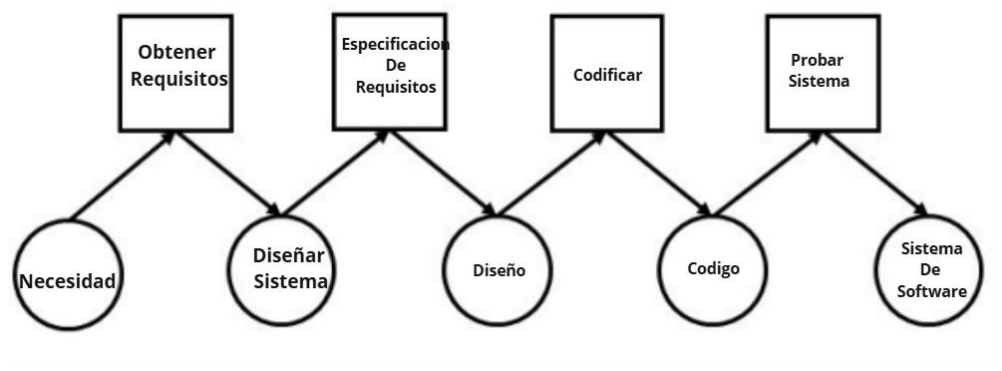

Ilustración 1 Grafico Pregunta 2,6

Necesidad, obtener requisitos, diseñar sistema, especificación de requisitos, diseño, codificar, código, probar, sistema de software.

Obtener requisitos, diseñar sistema, especificación de requisitos, diseño, codificar, código, probar, sistema de software, necesidad.

Necesidad, especificación de requisitos, obtener requisitos, diseñar sistema,

diseño, codificar, código, probar, sistema de software.

Justificación: El desarrollo de un sistema comienza con la identificación de una necesidad, que motiva la creación del software. A partir de esa necesidad se procede a obtener y especificar los requisitos, donde se analizan y documentan las funciones que el sistema debe cumplir. Luego se realiza el diseño del sistema, donde se define la arquitectura y la forma en que se implementará la solución.

Después de esta etapa se pasa a la codificación, donde los desarrolladores escriben el código que implementa el diseño. Posteriormente se realizan pruebas para verificar que el sistema funcione correctamente y cumpla con los requisitos definidos. Finalmente, como resultado de todo este proceso, se obtiene el sistema de software terminado.

7. Los modelos de calidad del producto software y los modelos de calidad del proceso de desarrollo de Software son:

ISO/IEC 25010 producto software.

ISO/IEC 20000 producto software.

ISO 9126 proceso de desarrollo.

Justificación: La norma ISO/IEC 25010 define a la Eficiencia de desempeño como una de las ocho características principales de calidad, mientras que la seguridad y la fiabilidad también poseen este mismo rango jerárquico y no el de sus características. Esta clasificación es fundamental ya que el modelo categoriza la calidad del producto específicamente en características y sus características para evaluar atributos no funcionales, situando al desempeño junto a elementos como la adecuación funcional, usabilidad y mantenibilidad dentro del esquema principal del estándar (Mauricio_Costanzo, 2020).

8. La norma ISO/IEC 25010 identifica características y subccaracterísticas de calidad del producto software. La opción correcta es:

Seguridad es subcaracterística.

Eficiencia de desempeño es característica.

Fiabilidad es subcaracterística.

Justificación:

La norma ISO/IEC 25010 define un modelo de calidad estructurado en características y subccaracterísticas. En este modelo, la eficiencia de desempeño es una característica principal, mientras que la seguridad y la fiabilidad también pertenecen a este nivel y no son subccaracterísticas.

Según Sommerville (2011), este estándar organiza la calidad del software en ocho características principales, cada una desglosada en atributos específicos para su evaluación. Por lo tanto, la opción correcta es: “Eficiencia de desempeño es característica”.

9. Según los roles y las responsabilidades correspondientes, qué opción es correcta.

Cambiar el código según las actividades de desarrollo estándar o las solicitudes

de cambio corresponde a un auditor.

Mantener la configuración del código corresponde a un configuration mánager.

Verificar los cambios y resolver conflictos corresponde a un desarrollador.

Justificación: En entornos de desarrollo modernos (especialmente con sistemas de control de versiones como Git), el desarrollador tiene un rol activo en:

- Verificar los cambios antes de integrarlos (pruebas, revisión de código).
- Resolver conflictos de código que surgen al combinar ramas (*merge conflicts*).

Aunque el *configuration manager* define políticas y supervisa la gestión de configuración, en la práctica diaria los desarrolladores son quienes resuelven directamente los conflictos y validan los cambios en el código.

10. Según los objetivos con la correspondiente fase del proceso unificado de Desarrollo, ¿qué opción es la correcta?

Inicio establece el alcance del proyecto y las condiciones de los límites, criterios de aceptación, lo que se necesita en el producto y lo que no.

Transición completa el análisis, diseño, desarrollo y prueba.

Construcción logra versiones de código útiles tan rápido como se pueda.

Justificación: la fase de Inicio en el Proceso Unificado tiene como objetivo principal:

- Definir el alcance del proyecto
- Establecer los requisitos iniciales
- Determinar qué se incluye y qué no en el sistema

TechLib. (2023). *Rational Unified Process (RUP): definición y explicación*. https://techlib.net/techedu/rational-unified-process-rup/

4. CONCLUSIONES

Se comprendieron los fundamentos de la ingeniería de software, reconociendo su importancia en el desarrollo de sistemas eficiente y productivos.

El desarrollo de las actividades de los capítulos 1 y 2 facilitó la aplicación de los conocimientos teóricos, fortaleciendo la comprensión de conceptos importantes.

5. RECOMENDACIONES

Mantenerse actualizado sobre las tendencias y avances en la ingeniería de software, especialmente en metodologías ágiles y tecnologías emergentes.

Complementar el aprendizaje teórico con herramientas y ejercicios prácticos que permitan una mejor comprensión de los conceptos aprendidos.

6.BIBLIOGRAFÍA

Mauricio_Costanzo. (2020, May 3). *Atributos de calidad de un producto de software [Online forum post]*. https://platzi.com. https://platzi.com/blog/atributos-de-calidad-de-un-producto-de-software/

Somerville, I. (2011). Software Engineering 9. Obtenido de https://ifs.host.cs.standrews.ac.uk/Books/SE9/index.htm

Pressman, R. 7. (2005). (M. Hill, Editor) Obtenido de

https://www.mlsu.ac.in/econtents/16_EBOOK7th_ed_software_engineering_a_practitioners_approach_by_roger_s._pressman_.pdf

TechLib. (2023). *Rational Unified Process (RUP): definición y explicación*. https://techlib.net/techedu/rational-unified-process-rup/

7. ANEXOS (opcionales)

8.TABLA DE ESTUDIANTES

GRUPO 1	
Nro.	Apellidos y Nombres
1	AYALA EATAN (Líder de Grupo)
2	CATOTA JOHAN
3	CUICHAN ESTEBAN
4	CAIZALUISA KATERINE

Tabla 2 Tabla Estudiantes